

Rotas da API

Controle Simplificado dos Motoristas Agregados

Parceiro acadêmico: Newelog
Equipe RubyFox · FATEC São José dos Campos
Gerado em 23/10/2026

Rotas da API

Documentação dos endpoints expostos pelo sistema. Toda chamada passa pelo **API Gateway**; o frontend e qualquer cliente externo nunca falam direto com os microsserviços.

- ms-usuarios: autenticação e cadastro de usuários
- ms-frota: motoristas, veículos e agregados
- ms-operacoes: importação de manifestos, viagens e dashboard
- Gateway: status dos serviços

Endereço base

Ambiente	URL base
Local	`http://localhost:8080/api`

O frontend lê a URL da variável `VITE_API_URL`. Sem ela, usa o valor acima.

Roteamento no gateway

Prefixo	Microsserviço	Porta interna
`/api/auth/**`, `/api/usuarios/**`	ms-usuarios	8081
`/api/motoristas/**`, `/api/veiculos/**`, `/api/agregados/**`	ms-frota	8082
`/api/importacao/**`, `/api/viagens/**`, `/api/dashboard/**`, `/api/ranking/**`	ms-operacoes	8083
`/api/status`	o próprio gateway	8080

Autenticação

A API usa **JWT** (HMAC-256, emissor `ms-usuarios`, validade de **2 horas**).

1. Faça login em `POST /api/auth/login` e guarde o `token` da resposta.
2. Envie o token em todas as outras chamadas: Inclua o token no cabeçalho HTTP usando o esquema Bearer.

```
Authorization: Bearer <token>
```

O gateway valida o token antes de rotear. Com o token válido, ele injeta dois headers na requisição que segue para o microsserviço:

Header	Conteúdo
`X-User-Name`	e-mail do usuário (subject do token)
`X-User-Role`	perfil de acesso (`Operador` , `Gestor` ou `Administrador`)

> Qualquer `X-User-Name` ou `X-User-Role` enviado pelo cliente é **descartado** pelo gateway. Não adianta forjar esses headers.

Rotas públicas (sem token)

Rota	Motivo
`POST /api/auth/login`	login
`POST /api/usuarios`	cadastro, aberto provisoriamente para criar o primeiro Administrador

Perfis de acesso

Perfil	Descrição
`Operador`	uso operacional (importação e consultas)
`Gestor`	acompanhamento gerencial (dashboard e indicadores)
`Administrador`	tudo acima, mais cadastro de usuários e status dos serviços

Formato de erro

Os erros tratados pela aplicação retornam JSON com a chave `erro` :

```
{ "erro": "Token invalido ou expirado" }
```

Status	Quando acontece
`400 Bad Request`	parâmetro inválido (ex.: `mesReferencia` fora do formato `AAAA-MM`) ou campo obrigatório ausente
`401 Unauthorized`	token ausente, inválido ou expirado; ou e-mail/senha incorretos no login
`403 Forbidden`	usuário inativo, ou perfil sem permissão para a rota
`404 Not Found`	recurso não encontrado
`409 Conflict`	conflito de estado (e-mail já cadastrado, importação já executada)

Status	Quando acontece
`422 Unprocessable Entity`	falha ao gravar a importação

Alguns `400` e `404` voltam **sem corpo**. Nesses casos, o status já basta.

Rotas do próprio gateway

`GET /api/status`

Consulta o `/actuator/health` de cada microsserviço (timeout de 3 s). O health já leva em conta a conexão com o banco, então um serviço que responde na porta mas não alcança o banco aparece como `DOWN`.

Perfil exigido: `Administrador`. Outros perfis recebem `403`.

Resposta `200`:

```
{
  "geral": "DOWN",
  "servicos": [
    { "servico": "ms-usuarios",  "status": "UP",   "tempoMs": 42 },
    { "servico": "ms-frota",     "status": "UP",   "tempoMs": 38 },
    { "servico": "ms-operacoes", "status": "DOWN", "tempoMs": 3001, "detalhe": "Sem resposta" }
  ]
}
```

Limites

Item	Limite
Tamanho do arquivo CSV	10 MB (requisição até 11 MB)
Validade do token	2 horas
Página da validação de importação	1 a 200 linhas
Ranking de motoristas	1 a 50 posições

ms-usuarios

Autenticação, emissão de JWT e cadastro de usuários. Porta interna **8081**, dono da tabela `usuarios`.

Método	Rota	Autenticação	Descrição
`POST`	`/api/auth/login`	pública	Autentica e devolve o token JWT
`POST`	`/api/usuarios`	pública (provisório)	Cadastra um usuário

`POST /api/auth/login`

Autentica o usuário por e-mail e senha.

Body:

```
{
  "email": "gestor@newelog.com",
  "senha": "minhasenha"
}
```

Resposta `200`:

```
{
  "sucesso": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "nome": "Maria Souza",
  "perfil": "Gestor"
}
```

O token carrega as claims `sub` (e-mail), `nome`, `perfil`, `iss` (`ms-usuarios`) e `exp` (2 horas).

Erros:

Status	Corpo
`401`	`{ "sucesso": false, "mensagem": "E-mail ou senha incorretos." }`
`403`	`{ "sucesso": false, "mensagem": "Usuário inativo. Contate o adminstrador." }`

`POST /api/usuarios`

Cadastra um novo usuário. A senha chega em texto e é gravada com **BCrypt**, então criar usuário direto no banco não funciona: o login compara hashes.

> **Atenção:** a rota está aberta provisoriamente para permitir criar o primeiro Administrador. Ela deve ser restrita ao perfil `Administrador` assim que ele existir.

Body:

```
{
  "nome": "Maria Souza",
  "email": "maria@newelog.com",
  "senha": "1234",
  "perfilAcesso": "Gestor"
}
```

Campo	Regra
`nome`	obrigatório
`email`	obrigatório, único, gravado em minúsculas
`senha`	obrigatório, mínimo de 4 caracteres
`perfilAcesso`	`Operador`, `Gestor` ou `Administrador` (maiúsculas e minúsculas tanto faz)

Resposta `201`:

```
{
  "idUsuario": "3f1c2a8e-5b7d-4e9a-9c1f-2d6b8a0e4f11",
  "nome": "Maria Souza",
  "email": "maria@newelog.com",
  "perfilAcesso": "Gestor",
  "statusAtivo": true
}
```

A senha (nem o hash dela) nunca aparece na resposta.

Erros:

Status	Exemplo de corpo
`400`	`{ "erro": "Campo obrigatorio: email" }`
`400`	`{ "erro": "A senha precisa de ao menos 4 caracteres" }`
`400`	`{ "erro": "Perfil invalido: Chefe. Use um destes: Operador, Gestor, Administrador" }`
`409`	`{ "erro": "Ja existe usuario com o e-mail maria@newelog.com" }`

ms-frota

Cadastro e consulta de motoristas, veículos e agregados. Porta interna **8082**, dono das tabelas `motoristas`, `veiculos`, `agregados`, `categorias_veiculos`, `cidades` e `estados`.

Todas as rotas exigem `Authorization: Bearer <token>` quando chamadas pelo gateway.

Método	Rota	Descrição
`GET`	`/api/motoristas/{cpf}`	Busca motorista por CPF
`POST`	`/api/motoristas/resumos`	Nome e CPF de vários motoristas por id
`POST`	`/api/motoristas/lote`	Upsert em lote por CPF
`GET`	`/api/veiculos/{placa}`	Busca veículo por placa
`POST`	`/api/veiculos/placas`	Placa de vários veículos por id
`POST`	`/api/veiculos/modelos`	Modelo de vários veículos por id
`POST`	`/api/veiculos/lote`	Upsert em lote por placa
`GET`	`/api/agregados/{documento}`	Busca agregado por CPF/CNPJ
`POST`	`/api/agregados/lote`	Upsert em lote por documento

> As rotas `/lote`, `/resumos`, `/placas` e `/modelos` existem para uso **entre serviços**: o ms-operacoes as chama durante a importação e no dashboard. Um manifesto traz centenas de linhas, e uma chamada HTTP por motorista seria inviável.

Motoristas

`GET /api/motoristas/{cpf}`

Parâmetro	Tipo	Descrição
`cpf`	path	CPF com 11 dígitos, só números

Resposta `200`:

```
{
  "idMotorista": "a1b2c3d4-0000-4000-8000-000000000001",
  "nome": "JOAO DA SILVA",
  "cpf": "12345678901",
  "pis": "12345678901",
  "dataNascimento": "1985-03-12",
  "cep": "12220000",
  "logradouro": "Rua das Flores",
  "numero": "100",
  "bairro": "Centro",
  "idCidade": 3549904,
  "criadoEm": "2026-09-20T14:31:00"
}
```

Erros: `404` se o CPF não existir.

`POST /api/motoristas/resumos`

Recebe uma lista de ids e devolve nome e CPF de cada um.

Body:

```
["a1b2c3d4-0000-4000-8000-000000000001", "a1b2c3d4-0000-4000-8000-000000000002"]
```

Resposta `200`:

```
[
  { "id": "a1b2c3d4-0000-4000-8000-000000000001", "nome": "JOAO DA SILVA", "cpf": "12345678901" }
]
```

`POST /api/motoristas/lote`

Cria ou atualiza motoristas pelo CPF. Devolve o mapa `cpf → id` de **todos** os CPFs recebidos, tanto os criados agora quanto os que já existiam. Itens sem CPF são ignorados, sem derrubar o lote.

Body:

```
[
  {
    "cpf": "12345678901",
    "nome": "JOAO DA SILVA",
    "pis": "12345678901",
    "dataNascimento": "1985-03-12",
    "endereco": "Rua das Flores, 100",
    "cep": "12220000",
    "bairro": "Centro",
    "cidade": "São José dos Campos",
    "estado": "SP"
  }
]
```

`endereco`, `cidade` e `estado` chegam como texto livre do CSV. O ms-frota separa logradouro e número e resolve a cidade na tabela `cidades`.

Resposta `200`:

```
{ "12345678901": "a1b2c3d4-0000-4000-8000-000000000001" }
```

Veículos

`GET /api/veiculos/{placa}`

Resposta `200`:

```
{
  "idVeiculo": "b1b2c3d4-0000-4000-8000-000000000010",
  "placa": "ABC1D23",
  "marca": "Volvo",
  "modelo": "FH 540",
  "numeroFrota": "F-012",
  "capacidade": 27000.00,
  "nomeAgregado": "TRANSPORTES EXEMPLO LTDA",
  "statusVeiculo": "ATIVO",
  "criadoEm": "2026-09-20T14:31:00"
}
```

Erros: `404` se a placa não existir.

`POST /api/veiculos/placas`

Body: lista de ids de veículos. **Resposta `200`:** mapa `id → placa`.

```
{ "b1b2c3d4-0000-4000-8000-000000000010": "ABC1D23" }
```

`POST /api/veiculos/modelos`

Body: lista de ids de veículos. **Resposta `200`:** mapa `id → modelo`.

```
{ "b1b2c3d4-0000-4000-8000-000000000010": "FH 540" }
```

`POST /api/veiculos/lote`

Cria ou atualiza veículos pela placa. O manifesto traz só placa, capacidade e nome do agregado, então o cadastro nasce mínimo e é completado depois.

Body:

```
[
  { "placa": "ABC1D23", "capacidade": 27000.00, "nomeAgregado": "TRANSPORTES EXEMPLO LTDA" }
]
```

Resposta `200`: mapa `placa → id`.

```
{ "ABC1D23": "b1b2c3d4-0000-4000-8000-000000000010" }
```

Agregados

`GET /api/agregados/{documento}`

Parâmetro	Tipo	Descrição
`documento`	path	CPF (11 dígitos) ou CNPJ (14 dígitos), só números

Resposta `200`:

```
{
  "idAgregado": "c1b2c3d4-0000-4000-8000-000000000020",
  "documento": "12345678000199",
  "tipoPessoa": "PJ",
  "nome": "TRANSPORTES EXEMPLO LTDA",
  "pis": null,
  "regimeFiscal": "Simples Nacional",
  "criadoEm": "2026-09-20T14:31:00"
}
```

`tipoPessoa` é calculado pelo tamanho do documento: `PF` para CPF (11 dígitos) e `PJ` para CNPJ (14 dígitos).

Erros: `404` se o documento não existir.

`POST /api/agregados/lote`

Cria ou atualiza agregados pelo documento.

Body:

```
[
  { "documento": "12345678000199", "nome": "TRANSPORTES EXEMPLO LTDA", "pis": null, "regimeFiscal":
"Simples Nacional" }
]
```

Resposta `200`: mapa `documento → id`.

```
{ "12345678000199": "c1b2c3d4-0000-4000-8000-000000000020" }
```

ms-operacoes

Importação do CSV de manifestos, consulta de viagens e indicadores do dashboard. Porta interna **8083**, dono das tabelas `importacoes`, `viagens`, `viagem_custos`, `viagem_destinos`, `tipos_custo` e `controle_disponibilidade`.

Todas as rotas exigem `Authorization: Bearer <token>`.

Método	Rota	Descrição
`POST`	`/api/importacao/arquivos`	Etapa 1: recebe o CSV
`POST`	`/api/importacao/arquivos/{id}/validar`	Etapa 2: valida sem gravar
`POST`	`/api/importacao/arquivos/{id}/executar`	Etapa 3: grava as viagens
`GET`	`/api/importacao/arquivos`	Lista as importações
`GET`	`/api/importacao/arquivos/resumo`	Quantidade de importações por status
`GET`	`/api/importacao/arquivos/{id}`	Detalhe de uma importação
`GET`	`/api/importacao/arquivos/{id}/conteudo`	Baixa o CSV original
`GET`	`/api/importacao/arquivos/{id}/viagens`	Viagens gravadas por uma importação
`GET`	`/api/importacao/arquivos/{id}/motoristas`	Ids dos motoristas de uma importação
`GET`	`/api/viagens`	Viagens de um mês
`GET`	`/api/dashboard/indicadores`	Indicadores do mês e por motorista
`GET`	`/api/dashboard/indicadores-por-modelo`	Indicadores por modelo de veículo
`GET`	`/api/dashboard/ranking-motoristas`	Top N motoristas por número de viagens
`GET`	`/api/ranking/mensal`	Ranking mensal completo dos motoristas, com ordenação por coluna
`GET`	`/api/dashboard/motoristas/situacao`	Situação (disponível/indisponível) de cada motorista no mês

---

Importação de manifestos

A importação é feita **em etapas**. Enviar o arquivo não grava nada: as viagens só entram no banco no `/executar`. Antes disso, o usuário pode validar e revisar quantas vezes quiser.

```
POST /arquivos  ──▶  POST /{id}/validar  ──▶  POST /{id}/executar
 (AGUARDANDO)       (sem efeito no banco)      (CONCLUIDO)
```

Status de uma importação

Status	Significado
`AGUARDANDO`	arquivo recebido e guardado; nada foi lido nem gravado
`VALIDADO`	arquivo lido e conferido; ainda nada gravado
`EM_PROCESSAMENTO`	gravação em andamento
`CONCLUIDO`	viagens gravadas
`ERRO`	falha na leitura ou na gravação

`POST /api/importacao/arquivos`

Recebe o arquivo e devolve o id da importação. O arquivo não é lido aqui.

Content-Type: `multipart/form-data`

Campo	Tipo	Regra
`arquivo`	file	obrigatório, extensão `.csv`, até 10 MB

```
curl -X POST http://localhost:8080/api/importacao/arquivos \
  -H "Authorization: Bearer $TOKEN" \
  -F "arquivo=@manifestos_3tri.csv"
```

Resposta `201`:

```
{ "id": 42, "arquivoNome": "manifestos_3tri.csv", "status": "AGUARDANDO" }
```

Erros:

Status	Corpo
`400`	`{ "erro": "Arquivo vazio" }`
`400`	`{ "erro": "Formato nao aceito. Envie um arquivo .csv" }`

> O mês de referência não é informado no envio. A planilha da Newelog é trimestral, então o mês de cada viagem vem da coluna **Data** da própria linha.

`POST /api/importacao/arquivos/{id}/validar`

Lê o arquivo e devolve as colunas encontradas, as linhas com problema (paginadas, erros antes de pendências) e uma amostra das linhas válidas. **Não grava nada** e pode ser chamado quantas vezes for preciso.

Parâmetro	Tipo	Padrão	Regra
`id`	path		id da importação
`pagina`	query	`0`	≥ 0
`tamanho`	query	`50`	1 a 200
`severidade`	query	todas	`ERRO` ou `PENDENCIA`. Filtra só a lista; o resumo sempre conta o arquivo inteiro

Resposta `200`:

```
{
  "importacaoId": 42,
  "arquivoNome": "manifestos_3tri.csv",
  "totalLinhas": 1024,
  "linhasValidas": 1019,
  "linhasInvalidas": 2,
  "linhasComPendencia": 3,
  "colunasDetectadas": ["Manifesto", "Filial", "Data", "Motorista", "CPF", "..."],
  "mapeamento": [
    { "campoSistema": "MANIFESTO", "obrigatoria": true,  "encontrada": true,  "colunaArquivo":
"Manifesto" },
    { "campoSistema": "PIS",       "obrigatoria": false, "encontrada": false, "colunaArquivo": null }
  ],
  "pagina": { "numero": 0, "tamanho": 50, "totalElementos": 5 },
  "problemas": [
    {
      "numeroLinha": 17,
      "valores": { "CPF": "11.222.333/0001-81", "Data": "12/07/2026" },
      "problemas": [
        {
          "campo": "CPF",
          "coluna": "CPF",
          "severidade": "ERRO",
          "estado": "NAO_CONVERTE",
          "valorEncontrado": "11.222.333/0001-81",
          "valorEsperado": "CPF com 11 digitos e digito verificador valido",
          "mensagem": "A coluna 'CPF' na linha 17 nao esta no formato esperado: CPF invalido: 14
digitos, parece um CNPJ"
        }
      ]
    }
  ],
  "amostra": [
    {
      "numeroLinha": 2,
      "manifesto": 900123,
      "data": "2026-07-01",
      "mesReferencia": "2026-07",
      "motorista": "JOAO DA SILVA",
      "cpf": "12345678909",
      "agregado": "TRANSPORTES EXEMPLO LTDA",
      "veiculo": "ABC1D23",
      "destino": "Campinas",
      "valorFrete": 1850.00
    }
  ]
}
```


Como a severidade é decidida:

Nível da coluna	Célula vazia	Valor em formato errado
Obrigatória (`Manifesto` , `Data` , `CPF` , `Veículo`)	`ERRO`	`ERRO`
Importante (ex.: `Motorista` , `Agregado` , `Destino` , `Valor Frete`)	`PENDENCIA`	`PENDENCIA`

Nível da coluna	Célula vazia	Valor em formato errado
Opcional	aceita	`PENDENCIA`

Estados possíveis de uma célula: `OK`, `VAZIO`, `NAO_CONVERTE` e `AUSENTE_COMO_ZERO` (colunas em que o CSV usa `0,00` para dizer "não informado").

Regras de formato dos campos cadastrais:

Coluna	Regra (`valorEsperado`)
`CPF`	11 dígitos, não todos iguais, dígitos verificadores válidos
`CPF/CNPJ Agregado`	11 dígitos validados como CPF ou 14 validados como CNPJ
`PIS`, `PIS Agregado`	11 dígitos com dígito verificador válido
`CEP Motorista`	8 dígitos
`Veículo`, `Reboque 1`, `Reboque 2`, `Reboque 3`	placa antiga (`AAA9999`) ou Mercosul (`AAA9A99`); aceita hífen e minúsculas
`Estado Motorista`	sigla de uma das 27 UFs

Nos documentos (CPF, CNPJ, PIS e CEP) a célula só pode ter dígitos e máscara (`.` , `-` , `/` , espaços). Letras ou outros caracteres fazem a célula ser recusada, em vez de serem descartados. A `mensagem` diz o que veio e o que era esperado, por exemplo `CPF invalido: contem letras`, `CPF invalido: 14 digitos, parece um CNPJ`, `CPF invalido: digito verificador nao confere`, `Placa invalida: 'AB12345' fora do padrao AAA9999 ou AAA9A99`.

O arquivo só pode ser gravado quando `linhasInvalidas = 0`, existe ao menos uma linha válida e nenhuma coluna obrigatória está faltando (todo item de `mapeamento` com `obrigatoria: true` tem `encontrada: true`). Linhas só com pendência não bloqueiam.

Erros: `404` se a importação não existir; `400` para paginação inválida ou `severidade=OK`.

`POST /api/importacao/arquivos/{id}/executar`

Grava as viagens. É o **único** endpoint do fluxo que escreve no banco. Motoristas, veículos e agregados são criados ou atualizados no ms-frota pelos endpoints de lote. Manifestos que já existem no banco são ignorados, então reimportar o mesmo arquivo não duplica dados.

Resposta `200`:

```
{ "id": 42, "status": "CONCLUIDO", "linhasGravadas": 1019 }
```

Erros:

Status	Corpo
`404`	sem corpo, se a importação não existir
`409`	`{ "erro": "Esta importacao ja foi executada" }`
`422`	`{ "erro": "<motivo da falha>" }`

`GET /api/importacao/arquivos`

Lista todas as importações.

Resposta `200`:

```
[
  {
    "id": 42,
    "arquivoNome": "manifestos_3tri.csv",
    "dataImportacao": "2026-09-28T10:15:00",
    "status": "CONCLUIDO",
    "usuarioResponsavel": "operador@newelog.com",
    "totalLinhas": 1024,
    "linhasValidas": 1019,
    "linhasInvalidas": 0,
    "linhasGravadas": 1019
  }
]
```

O conteúdo binário do arquivo não aparece. Para obtê-lo, use `/{id}/conteudo`.

`GET /api/importacao/arquivos/resumo`

Resposta `200`:

```
{
  "totalImportacoes": 7,
  "porStatus": { "AGUARDANDO": 1, "VALIDADO": 0, "EM_PROCESSAMENTO": 0, "CONCLUIDO": 5, "ERRO": 1 }
}
```

`GET /api/importacao/arquivos/{id}`

Mesmo formato de um item da listagem. `404` se não existir.

`GET /api/importacao/arquivos/{id}/conteudo`

Baixa o CSV original como anexo (`Content-Disposition: attachment`). `404` se não existir.

`GET /api/importacao/arquivos/{id}/viagens`

Viagens gravadas por essa importação, no mesmo formato de `GET /api/viagens`. `404` se a importação não existir.

`GET /api/importacao/arquivos/{id}/motoristas`

Ids (UUID) dos motoristas presentes na importação. Nome e CPF ficam no ms-frota: consulte `POST /api/motoristas/resumos` com esses ids.

```
["a1b2c3d4-0000-4000-8000-000000000001", "a1b2c3d4-0000-4000-8000-000000000002"]
```

Viagens

`GET /api/viagens`

Parâmetro	Tipo	Regra
`mesReferencia`	query	obrigatório, formato `AAAA-MM`

```
GET /api/viagens?mesReferencia=2026-09
```

Resposta `200` (campos principais):

```
[
  {
    "idViagem": 1501,
    "idManifesto": 900123,
    "idMotorista": "a1b2c3d4-0000-4000-8000-000000000001",
    "idVeiculo": "b1b2c3d4-0000-4000-8000-000000000010",
    "idAgregado": "c1b2c3d4-0000-4000-8000-000000000020",
    "importacaoId": 42,
    "dataViagem": "2026-09-03",
    "mesReferencia": "2026-09",
    "filial": "SJC",
    "destino": "Campinas",
    "kmSaida": 120300,
    "kmChegada": 120480,
    "kgReal": 8200.00,
    "kgTaxado": 8500.00,
    "servicos": 12,
    "servicosFinalizados": 12,
    "nfs": 14,
    "valorFrete": 1850.00,
    "valorNf": 95000.00,
    "totalDespesas": 420.00,
    "saldoAPagar": 1430.00,
    "status": "Finalizado",
    "usuarioManifesto": "fulano"
  }
]
```

A entidade também traz `reboque1..3`, `m3`, `coletas`, `entregas`, `despachos`, `retiradas`, `coletasReversa`, `percentualEfetividade`, `percentualAprovVeiculo`, `valorFretes`, `valeFrete`, `saldoDespesas`, `classificacao`, `observacoesOperacionais` e `criadoEm`.

Os campos de valor (`valorFrete`, `valorFretes`, `valeFrete`, `valorNf`, `totalDespesas`, `saldoDespesas`, `saldoAPagar`) só vêm para os perfis Gestor e Administrador. Para os demais, saem da resposta.

Erros: `400` se `mesReferencia` estiver fora do formato.

Dashboard

Valores monetários em R$; percentuais de 0 a 100.

- **Receita e custo:** valor do frete = soma de `valor_fretes` (frete dos CT-e cobrado do cliente); custo total = soma de `vale_frete` (total pago ao agregado). Só entra a viagem apurável, com os dois valores maiores que zero; as demais contam como viagem e dia em operação.
- **Sem base para a conta:** utilização e disponibilidade com divisão por zero retornam `0`. Já `rentabilidadeMediaViagem` (sem viagem apurável) e `margem` (sem frete) saem da resposta: zero diria que o resultado foi zero, quando não há dado.
- **Perfil:** `valorFrete`, `custoTotal`, `rentabilidade`, `rentabilidadeMediaViagem` e `margem` só vêm para Gestor e Administrador, em todos os endpoints desta seção e em `/api/ranking/mensal`.

- **Nomes:** os mesmos campos têm o mesmo nome em todos esses endpoints.

`GET /api/dashboard/indicadores`

Totais do mês e uma linha por motorista, ordenada por rentabilidade.

Parâmetro	Tipo	Regra
`mesReferencia`	query	obrigatório, `AAAA-MM` com mês de 01 a 12

Resposta `200`:


```
{
  "mesReferencia": "2026-09",
  "diasNoMes": 30,
  "diasDisponiveis": 26,
  "totais": {
    "motoristas": 48,
    "numeroViagens": 612,
    "utilizacaoMedia": 71.40,
    "valorFrete": 1203450.00,
    "custoTotal": 845000.00,
    "rentabilidade": 358450.00,
    "rentabilidadeMediaViagem": 585.70,
    "margem": 29.78
  },
  "motoristas": [
    {
      "motoristaId": "a1b2c3d4-0000-4000-8000-000000000001",
      "nome": "JOAO DA SILVA",
      "numeroViagens": 22,
      "diasOperacao": 20,
      "diasDisponiveis": 26,
      "utilizacao": 76.92,
      "disponibilidade": 86.67,
      "valorFrete": 41200.00,
      "custoTotal": 27800.00,
      "rentabilidade": 13400.00,
      "rentabilidadeMediaViagem": 609.09,
      "margem": 32.52
    }
  ]
}
```


Fórmulas:

Indicador	Cálculo
Utilização	dias em operação ÷ dias disponíveis × 100
Disponibilidade	dias disponíveis ÷ dias do mês × 100
Rentabilidade	valor do frete − custo total

Indicador	Cálculo
Rentabilidade média por viagem	rentabilidade ÷ número de viagens apuráveis
Margem	rentabilidade ÷ valor do frete × 100
Utilização média (totais)	soma dos dias em operação ÷ (dias disponíveis × motoristas) × 100

Erros: `400` se `mesReferencia` for inválido.

`GET /api/dashboard/indicadores-por-modelo`

Os mesmos indicadores agrupados por modelo de veículo.

Parâmetro	Tipo	Regra
`mesReferencia`	query	obrigatório, `AAAA-MM`

Resposta `200`:

```
[
  {
    "modelo": "FH 540",
    "numeroVeiculos": 12,
    "numeroViagens": 180,
    "diasOperacao": 240,
    "utilizacao": 76.90,
    "valorFrete": 402000.00,
    "custoTotal": 280000.00,
    "rentabilidade": 122000.00,
    "margem": 30.35
  }
]
```

`GET /api/dashboard/ranking-motoristas`

Os N motoristas com mais viagens no mês, com o tipo do veículo usado na viagem mais longa.

Viagem mais longa (mesma regra de `/api/ranking/mensal`): a distância é o `km_saida`, que no manifesto já é a distância da rota (o `km_chegada` vem sempre 0). Em empate, ou sem nenhum km informado, fica a viagem mais recente. Veículo fora do cadastro aparece como `"Não informado"`.

Parâmetro	Tipo	Padrão	Regra
`mesReferencia`	query		obrigatório, `AAAA-MM`
`limite`	query	`5`	1 a 50

```
GET /api/dashboard/ranking-motoristas?mesReferencia=2026-09&limite=5
```

Resposta `200`:

```
[
  {
    "posicao": 1,
    "motoristaId": "a1b2c3d4-0000-4000-8000-000000000001",
    "nome": "JOAO DA SILVA",
    "cpf": "12345678901",
    "numeroViagens": 22,
    "tipoVeiculo": "FH 540",
    "distanciaMaximaKm": 480
  }
]
```

Um mês sem viagens devolve `[]`. **Erros:** `400` para parâmetros inválidos.

`GET /api/ranking/mensal`

Todos os motoristas do mês, com os mesmos indicadores de `/api/dashboard/indicadores` (mesmo cálculo e mesmos nomes, sem a margem) mais o tipo de veículo da viagem mais longa.

Parâmetro	Tipo	Padrão	Regra
`mesReferencia`	query		obrigatório, `AAAA-MM` com mês de 01 a 12
`ordenarPor`	query	`rentabilidade`	`nome`, `tipoVeiculo`, `numeroViagens`, `disponibilidade`, `utilizacao`, `valorFrete`, `custoTotal`, `rentabilidade` ou `rentabilidadeMediaViagem`
`direcao`	query	`desc`	`asc` ou `desc`

Valor ausente fica no fim nas duas direções. Para quem não vê dado financeiro, ordenar por um campo financeiro cai em `numeroViagens`.

Resposta `200`:

```
[
  {
    "motoristaId": "a1b2c3d4-0000-4000-8000-000000000001",
    "nome": "JOAO DA SILVA",
    "tipoVeiculo": "FH 540",
    "numeroViagens": 22,
    "disponibilidade": 100.00,
    "utilizacao": 76.92,
    "valorFrete": 41200.00,
    "custoTotal": 27800.00,
    "rentabilidade": 13400.00,
    "rentabilidadeMediaViagem": 609.09
  }
]
```

Erros: `400` para parâmetros inválidos.

`GET /api/dashboard/motoristas/situacao`

Situação de cada motorista no mês, a partir da tabela `controle_disponibilidade`. A importação (fim do `/executar`) grava os **dias disponíveis e em operação** de cada motorista; a consulta só lê essa tabela, sem tocar em `viagens`.

A tabela não tem colunas para utilização, faixa e situação (o schema do Supabase é fixo), então esses três campos são **calculados na consulta** a partir dos dias gravados. Por isso, para mudar os cortes basta reiniciar o ms-operacoes, sem reimportar.

Escopo: só motoristas com viagem no mês. Quem não rodou nenhum dia não aparece.

Parâmetro	Tipo	Regra
`mesReferencia`	query	obrigatório, `AAAA-MM` com mês de 01 a 12
`situacao`	query	opcional: `DISPONIVEL`, `INDISPONIVEL` ou `SEM_DIAS_DISPONIVEIS`

```
GET /api/dashboard/motoristas/situacao?mesReferencia=2026-09&situacao=DISPONIVEL
```

Resposta `200`:

```
[
  {
    "motoristaId": "a1b2c3d4-0000-4000-8000-000000000001",
    "nome": "JOAO DA SILVA",
    "diasDisponiveis": 30,
    "diasOperacao": 9,
    "utilizacao": 30.00,
    "faixaUtilizacao": "BAIXA",
    "situacao": "DISPONIVEL",
    "aviso": null
  }
]
```

Ordem: `DISPONIVEL` primeiro, depois menor utilização, depois nome. Sem resultado devolve `[]` (a mensagem "Nenhum motorista disponível com os filtros aplicados." é do front).

Regras do tratamento:

Item	Regra
Utilização	dias em operação ÷ dias disponíveis × 100, 2 casas (HALF_UP). Pode passar de 100
Dias em operação	dias distintos com viagem no mês, somando todas as importações
Dias disponíveis	mesmo valor de `/indicadores` (hoje, todos os dias do mês; PENDENTE do cliente)
`BAIXA`	utilização < 50 → `DISPONIVEL`
`INTERMEDIARIA`	50 ≤ utilização < 80 → `DISPONIVEL`

Item	Regra
`ALTA`	utilização ≥ 80 → `INDISPONIVEL`
Dias disponíveis ≤ 0	sem faixa, `utilizacao: null`, situação `SEM_DIAS_DISPONIVEIS` e `aviso: "Motorista sem dias disponíveis registrados no período."`

Os cortes ficam em `src/main/resources/situacao-utilizacao.properties` (`situacao.utilizacao.corte-baixa=50` e `situacao.utilizacao.corte-alta=80`). Exatamente 50 é `INTERMEDIARIA`; exatamente 80 é `ALTA`. Variável de ambiente sobrescreve o arquivo (`SITUACAO_UTILIZACAO_CORTEBAIXA`, `SITUACAO_UTILIZACAO_CORTEALTA`). Valor ausente, negativo ou corte baixo maior que o alto impede a subida do ms-operacoes, com o nome da propriedade na mensagem.

A cada importação, os dias dos meses presentes no arquivo são recalculados a partir de todas as viagens do mês no banco. Se o recálculo falhar, a importação continua `CONCLUIDO` (as viagens já foram gravadas) e o erro fica no log; a próxima importação do mês recalcula.

Erros: `400` se `mesReferencia` ou `situacao` forem inválidos.